

拟态语言技术手册

正则引擎

之恪提案

2026 年 3 月 15 日

目录

1 基础概念	1
1.1 字符与字符集	1
1.2 字符串	1
1.3 基本运算	1
1.4 非确定性有限状态自动机 (NFA)	2
1.4.1 NFA 的组成成分	2
1.4.2 形式化定义	3
1.4.3 工作原理	3
1.4.4 扩展转移函数	3
1.4.5 语言接受性	3
1.5 匹配规则	4
1.5.1 贪婪型量词匹配	4
1.5.2 懒惰型量词匹配	4
1.5.3 占有型量词匹配	5
1.5.4 三种匹配规则对比	5
1.6 转义字符	5
1.6.1 转义字符的基本概念	5
1.6.2 反斜杠的转义规则	5
1.6.3 ASCII 码中的转义字符	6
1.6.4 八进制和十六进制转义	6
1.6.5 无法转义字符的处理	6
1.7 表达式中的字符标记	7
2 正则结构的构建	8
2.1 状态节点	8
2.2 构建规则	8
2.3 状态节点的结构	9
2.4 虚转移节点	10
2.5 正则表的连接	10
2.6 编译时正则表	10
2.7 编译时正则表的首尾相接	11
2.8 状态节点的类型	12
2.9 计算有限正则表的长度	12
2.10 反运算修饰的正则结构	13

1 基础概念

1.1 字符与字符集

字符本质上是一个特定的自然数数值，这个数值称为字符的编码。我们称所有字符的集合为字符集 (或字母表)，使用符号 Σ 表示。由于字符的数量是有限的，因此有：

$$\exists \text{CHARACTER_MAX}, \forall c \in \Sigma, c \leq \text{CHARACTER_MAX}$$

故我们可以使用数胞 $NC_{\text{CHARACTER_MAX}+1}$ 存储字符，且字符集中的所有字符的编码都可以使用这个数胞表示。

1.2 字符串

字符是字母表中的元素，字符串是由字符组成的有限序列。

我们可以将字符串定义为字符编码的有限序列，其长度即为序列中字符的个数。更严谨地说：

设 Σ 为给定的字符集， Σ^* 表示由 Σ 中字符构成的所有有限序列的集合。一个字符串 s 是 Σ^* 中的一个元素，可表示为

$$s = c_1 c_2 \dots c_n, \quad c_i \in \Sigma \ (i = 1, \dots, n)$$

其中 $n \in \mathbb{N}$ 称为字符串的长度，记为 $|s|$ 。

形式化地，可定义长度为 n 的字符串为一个函数

$$s : \{1, 2, \dots, n\} \rightarrow \Sigma$$

当 $n = 0$ 时得到唯一的空串，通常记为 ϵ ，并且规定 $|\epsilon| = 0$ 。

并且规定红色的 $\cdot$ 可以代表任意字符。

这样， $|s|$ 就是从字符串到自然数集的映射

$$|\cdot| : \Sigma^* \rightarrow \mathbb{N}$$

其中 $\cdot$ 为任意字符串

1.3 基本运算

Σ^* 的子集 U 和 V 的连接 (积) 定义为

$$UV = \{\alpha\beta \mid \alpha \in U \wedge \beta \in V\}$$

V 自身的 n 次积记为

$$V^n = \underbrace{VV \dots V}_{\text{其中有 } n \text{ 个 } V}$$

$$V^0 = \varepsilon$$

V^* 是 V 的闭包，即

$$V^* = V^0 \cup V^1 \cup V^2 \cup \dots = \bigcup_{i=0}^{\infty} V^i$$

$U|V$ 是或运算，即

$$U \cup V$$

$\sim V$ 是反运算，在 V 的长度大于 0 时，意为与 V 长度相同，但不完全与 V 相同的任意字符串，即

$$|\sim V| = |V| \wedge \sim V \neq V$$

在长度为 0 时，等价于 ε 。

1.4 非确定性有限状态自动机 (NFA)

非确定性有限状态自动机是一种用于描述正则语言的计算模型。NFA 在状态转移过程中允许存在不确定性，即从一个状态读取某个字符后可能转移到多个不同的状态。

1.4.1 NFA 的组成成分

NFA 由五个基本成分组成：

1. **状态集合**：一个有限的状态集合，通常用 Q 表示。每个状态代表自动机在识别过程中的一个特定配置。
2. **输入字母表**：一个有限的输入符号集合，通常用 Σ 表示。自动机只能识别属于该字母表的字符序列。
3. **状态转移函数**：定义状态之间转移关系的函数，通常用 δ 表示。对于 NFA，转移函数的形式为：

$$\delta : Q \times \Sigma \rightarrow 2^Q$$

其中 2^Q 表示 Q 的幂集（即 Q 的所有子集的集合）。这意味着对于每个状态和输入符号的组合，NFA 可以转移到多个可能的状态。

4. **初始状态**：自动机开始运行时的起始状态，通常用 $q_0 \in Q$ 表示。NFA 可能允许多个初始状态，此时初始状态为一个状态集合。
5. **接受状态集合**：也称为终结状态集合，通常用 $F \subseteq Q$ 表示。当自动机处理完整个输入字符串后，如果最终处于某个接受状态，则认为该字符串被自动机接受。

1.4.2 形式化定义

一个 NFA 可以形式化地定义为一个五元组：

$$M = (Q, \Sigma, \delta, q_0, F)$$

其中：

- Q 是有限状态集合
- Σ 是有限输入字母表
- $\delta : Q \times \Sigma \rightarrow 2^Q$ 是状态转移函数
- $q_0 \in Q$ 是初始状态
- $F \subseteq Q$ 是接受状态集合

1.4.3 工作原理

NFA 通过以下方式处理输入字符串：从初始状态开始，依次读取输入字符串中的每个字符，根据转移函数确定可能转移到的状态集合。由于转移的非确定性，NFA 在每一步都可能处于多个状态的叠加中。当处理完整个输入字符串后，如果当前状态集合中包含至少一个接受状态，则该输入字符串被 NFA 接受。

1.4.4 扩展转移函数

为了处理整个字符串而不仅仅是单个字符，需要定义扩展转移函数 $\hat{\delta} : Q \times \Sigma^* \rightarrow 2^Q$ ：

- 对于空字符串 ϵ ，有 $\hat{\delta}(q, \epsilon) = \{q\}$
- 对于非空字符串 $w = xa$ （其中 $x \in \Sigma^*$, $a \in \Sigma$ ），有：

$$\hat{\delta}(q, w) = \bigcup_{p \in \hat{\delta}(q, x)} \delta(p, a)$$

1.4.5 语言接受性

NFA M 接受的语言定义为所有能够使 NFA 从初始状态转移到某个接受状态的字符串集合：

$$L(M) = \{w \in \Sigma^* \mid \hat{\delta}(q_0, w) \cap F \neq \emptyset\}$$

NFA 的特例

在具体的实现过程中，我们偏向于实现 NFA 的特例，而不是完整的 NFA。

- 状态转移函数只接受单个字符。
- 初始状态只有一个。
- 不建议设计从一个状态读取某个字符后可能转移到多个不同的状态的 NFA，因为我们打算使用占有型量词进行匹配，相关内容之后会介绍。

1.5 匹配规则

1.5.1 贪婪型量词匹配

贪婪型量词匹配是正则表达式中最基本的匹配策略。采用贪婪匹配的量词会尽可能多地匹配字符，只有在后续匹配失败时才会回溯并减少匹配的字符数量。

特点：

- 最大长度匹配：优先尝试匹配尽可能多的字符
- 回溯机制：当后续模式无法匹配时，会回溯并尝试较短的匹配
- 默认行为：多数正则表达式引擎默认采用贪婪匹配

示例：对于模式 $a \cdot * b$ 和字符串 "aabab"

- 贪婪匹配结果："aabab"
- 匹配过程： $\cdot *$ 首先匹配到字符串末尾，然后回溯直到找到最后一个 b

1.5.2 懒惰型量词匹配

懒惰型量词匹配（也称为懒惰匹配或最小匹配）与贪婪匹配相反，它会尽可能少地匹配字符，只有在后续模式需要时才会增加匹配的字符数量。

特点：

- 最小长度匹配：优先尝试匹配尽可能少的字符
- 渐进扩展：当后续匹配失败时，逐步增加匹配范围
- 显式指定：需要通过特定语法明确指定

示例：对于模式 $a \cdot * b$ 和字符串 "aabab"

- 懒惰匹配结果："aab"
- 匹配过程： $\cdot *$ 在遇到第一个 b 时就停止匹配

1.5.3 占有型量词匹配

占有型量词匹配类似于贪婪匹配，但一旦匹配成功就不会进行回溯，即使后续模式匹配失败也不会释放已匹配的字符。

特点：

- 无回溯匹配：匹配成功后不会释放已占有的字符
- 性能优化：避免回溯过程，提高匹配效率
- 严格匹配：一旦匹配失败就立即返回失败结果

示例：对于模式 $a \cdot * b$ 和字符串 "aabab"

- 占有匹配结果：匹配失败
- 匹配过程： $\cdot *$ 匹配所有剩余字符，不释放给后面的 b 匹配

1.5.4 三种匹配规则对比

特性	贪婪匹配	懒惰匹配	占有匹配
匹配原则	尽可能多	尽可能少	尽可能多（不回溯）
回溯行为	有回溯	有回溯	无回溯
性能	中等	中等	较高
适用场景	默认情况	需要精确匹配	性能敏感场景

这三种匹配规则为正则表达式提供了灵活的匹配策略，可以根据具体需求选择合适的匹配方式，在匹配精度和性能之间取得平衡。但是在拟态语言中，为了安全性和效率，我们默认只使用 NFA 的特例配上占有匹配。

1.6 转义字符

1.6.1 转义字符的基本概念

转义字符是一种特殊的字符序列，用于表示那些在正则表达式中具有特殊含义的字符的字面值，或者表示不可见的控制字符。转义字符通常以反斜杠 (\) 开头，后跟一个或多个字符。

1.6.2 反斜杠的转义规则

反斜杠在正则表达式中具有作用：**赋予普通字符特殊含义**
当反斜杠置于某些普通字符之前时，会赋予其特殊的匹配功能。

示例：

- $\backslash \cdot$ 匹配通配符（赋予字面字符 $\cdot$ 特殊含义）
- $\backslash n$ 匹配换行符（赋予字面字符 n 特殊含义）

1.6.3 ASCII 码中的转义字符

以下是正则表达式中常用的 ASCII 转义字符：

转义序列	ASCII 值	含义
\a	0x07	响铃符 (BEL)
\b	0x08	退格符 (BS)
\t	0x09	水平制表符 (TAB)
\n	0x0A	换行符 (LF)
\v	0x0B	垂直制表符 (VT)
\f	0x0C	换页符 (FF)
\r	0x0D	回车符 (CR)
\e	0x1B	转义符 (ESC)
\0	0x00	空字符 (NUL)

1.6.4 八进制和十六进制转义

除了预定义的转义序列外，还可以使用数值转义：

- **八进制转义：**\ddd (1-3 位八进制数字)
- **十六进制转义：**\xhh (2 位十六进制数字) 或 \x{hhhh} (1-4 位十六进制数字)

示例：

- \101 匹配字符'A' (八进制 65)
- \x41 匹配字符'A' (十六进制 41)

1.6.5 无法转义字符的处理

当反斜杠后接的字符不属于任何预定义的转义序列，且该字符本身也不具有特殊含义时，反斜杠将被忽略，后面的字符将按原义进行匹配。

示例：

- \a 匹配响铃符 (有效转义)
- \q 匹配字符q (无效转义，按原义处理)
- \@ 匹配字符@ (非元字符，按原义处理)

这种设计确保了向后兼容性，即使未来引入新的转义序列，现有的模式也不会被破坏。在实际使用中，建议只对具有特殊含义的字符进行转义，以避免混淆。

1.7 表达式中的字符标记

首先，对正则表达式 R 中的每个字面字符和操作符（元字符）出现进行标记，使每个出现具有唯一的位置索引。假设正则表达式 R 由字母表 Σ 中的字符、特殊字符（如操作符 $|, *, (,)$ 等，都使用红色标记，且都需要转义）和空表达式 ε 、通配符 $\cdot$ 组成。我们只关注字面字符和特殊字符的出现。（之后的红色字符默认代表一个转移字符后继一个字面字符形成的特殊字符）

定义 P 为 R 中所有字面字符出现的集合。每个出现 $p \in P$ 具有：

- 字符 $c(p) \in \Sigma$ （即该位置对应的字面字符）。
- 唯一位置索引 $i(p)$ （默认从左到右按从 0 开始的顺序编号，特殊字符也需要占用编号）。

例如，对于正则表达式 $R = \text{"aa* b"}$ ，字面字符出现为：位置 0 的'a'、位置 1 的'a'、位置 2 的'*'、位置 3 的'b'。因此， $P = \{p_0, p_1, p_2\}$ ，其中 $c(p_0) = 'a', c(p_1) = 'a', c(p_2) = 'b', i(p_0) = 0, i(p_1) = 1, i(p_2) = 3$ 。

注意，对于反斜杠与其紧接字符形成的转义或原义组合，将其视为一个字面字符。反运算紧接一个普通的字面字符，将其视为一个反字面字符，是字面字符的一种特例。

2 正则结构的构建

2.1 状态节点

基于标记的字符出现，定义状态节点集合 Q 。每个状态节点对应一个字符出现或用于控制流程（如开始状态）。状态节点有唯一 ID 和类型。

- 令 $Q = \{q_p \mid p \in P\} \cup \{q_0\}$ ，其中 q_0 是开始状态。
- 每个状态节点 q_p （对于 $p \in P$ ）有三个成员：
 - 转移字符：即 $c(p)$ ，表示进入该状态时需要匹配的字符。
 - 状态类型：例如普通字符匹配，用于非反字符 $c(p)$ 。通配符匹配。空转移，用于 ε 转移的状态。反字符匹配，用于反字符 $c(p)$ 。
 - 连接的其他状态节点引用：即从该状态出发的转移目标列表。
- 状态节点 q_0 是开始状态，没有转移字符（空转移）。

2.2 构建规则

接下来我们需要将正则表达式字面量构建成 NFA，我们需要定义一些构建规则，以指导构建的过程。

对于表达式 $R_1 = \varepsilon$ 和任意的 R_2 , $R_1 \cdot R_2 = R_2$; $R_2 \cdot R_1 = R_2$ 。

假设存在表达式 R_0 :

- 设 $q_{start}^{R_0}$ 为分支 R_0 的起始状态集合
- 设 $q_{end}^{R_0}$ 为分支 R_0 的结束状态集合

对于表达式 $R = aA$ ，其中 a 为字符， A 为表达式，有 $q_{start}^R = a$ ；对于表达式 $R = Aa$ ，其中 a 为字符， A 为表达式，有 $q_{end}^R = a$ 。

对于非空表达式 S ，若有 $R_0 = (S)$ ，则 $q_{start}^{R_0} = q_{start}^S$ ，且 $q_{end}^{R_0} = q_{end}^S$ 。

对于非空表达式 P, S ，若有 $R_0 = P \cdot S$ ，则 $\forall q_s \in q_{start}^S, \forall q_e \in q_{end}^P, \exists \delta(q_e, c(q_s)) = q_s$ 。

对于或运算 $R_1 | R_2 | \dots | R_n$ ，定义以下构建规则：

状态定义：

- 对于表达式 $R = R_1 | R_2 | \dots | R_n$ ，且 $\forall i$ 满足 $1 \leq i \leq n, \forall R_i \neq \varepsilon$ ，有 $q_{start}^{R_i} \subseteq q_{start}^R$
- 对于表达式 $R = R_1 | R_2 | \dots | R_n$ ，且 $\forall i$ 满足 $1 \leq i \leq n, \forall R_i \neq \varepsilon$ ，有 $q_{end}^{R_i} \subseteq q_{end}^R$
- 对于表达式 $R = P | \varepsilon | S$ ，或者表达式 $R = \varepsilon | S$ ，或者表达式 $R = P | \varepsilon$ ，有 $((\forall q_p, \forall q_{R_s} \in q_{start}^R, \exists \delta(q_p, c(q_{R_s})) = q_{R_s}), (\forall q_s, \forall q_{R_e} \in q_{end}^R, \exists \delta(q_{R_e}, c(q_s)) = q_s)) \Rightarrow (\exists \delta(q_p, c(q_s)) = q_s)$ 。

- 对于表达式 $R = R_1 | R_2 | \dots | R_n$ ，如果这个或运算的作用域为表达式的顶层，设 q_0 为 NFA 的起始状态，则 $\forall q_s \in q_{start}^R, \exists \delta(q_0, c(q_s)) = q_s$

对于闭包运算 R_0^* ，定义以下构建规则：

闭包运算的转移函数定义：

- $(\epsilon)^* = \epsilon$ ，下文提及的 $R_0 \neq \epsilon$ 。
- 对于 $R = R_0^*$ ， $q_{start}^R = q_{start}^{R_0}$ ， $q_{end}^R = q_{end}^{R_0}$ 。
- $\forall q_{end}^{R_0}, \forall q_{start}^{R_0}, \exists \delta(q_{end}^{R_0}, c(q_{start}^{R_0})) = q_{start}^{R_0}$ 。
- $((\forall q_p, \forall q_{R_0s} \in q_{start}^{R_0}, \exists \delta(q_p, c(q_{R_0s})) = q_{R_0s}), (\forall q_s, \forall q_{R_0e} \in q_{end}^{R_0}, \exists \delta(q_{R_0e}, c(q_d)) = q_s)) \Rightarrow (\exists \delta(q_p, c(q_s)) = q_s)$ 。

2.3 状态节点的结构

定义 `NUMBER_OF_CHARACTER` 为字母表中所有字符的种类数量。

定义 `WORDSIZEVALUE` 为计算机一个字长的存储空间所使用的状态数。

定义 `TYPEVALUE` 为状态节点的状态类型种类数量。

定义以 `TYPEVALUE` 为状态数的数胞在结构体中后继以 `NUMBER_OF_CHARACTER` 为状态数的数胞刚好填充完以 `WORDSIZEVALUE` 为状态数的数胞所占的全部存储空间。

定义结构 `MST`，其第一个成员 `character` $\in NC_{\text{NUMBER_OF_CHARACTER}}$ ，第二个成员 `type` $\in NC_{\text{TYPEVALUE}}$ ，成员在存储空间中依次顺序存储。

定义结构 `ST`，其第一个成员 `index` $\in SNC_{\text{WORDSIZEVALUE}}$ ，第二个成员 `mst` $\in MST$ ，成员在存储空间中共享存储区域。

其中，`MST` 代表一个有实际字面意义的节点，它有确定的转移条件 `character` 和类型 `type`。在实际实现中，我们通过 `type` 的取值限定来区分一个状态节点 `ST` 是 `MST` 还是一个索引 `index`，但在这里没法表现区分的方法。区分的方法为：如果 `ST` 实例中 `index` 映射一个负数，那么这个 `ST` 就是一个 `MST`，否则是一个索引。也就是说，`type` 一旦取值，`index` 就必须是负数（映射的规则为 `index` 的值减去 $\lfloor \frac{\text{WORDSIZEVALUE}}{2} \rfloor + 1$ 所表示的整数）。至于为什么能这么实现，因为计算机的单字长存储空间的状态数为 `WORDSIZEVALUE`，而 `ST` 满足不只占一个字节的存储空间，因此，就算计算机中的存储空间全部用于存储 `ST`，数量也不会超过 `WORDSIZEVALUE` 的一半，进而，当 `ST` 是一个 `SNC` 索引时，永远不会溢出为负数，那么负数的索引就可以用于表示这个 `ST` 是一个 `MST` 了。

那么为什么需要状态节点 `ST` 有两种分类——`MST` 和索引呢？我们知道 `MST` 就是前文所说的状态节点的标准形式，转移到此状态所需要匹配的字符 `character` 以及状态节点的类型 `type`。但是，如果只有 `MST`，我们无法描述正则表中一个 `MST` 到另一个 `MST` 的转移路径（使用状态节点的线性数组表示正则表）。因此，我们需要额外的

一种类型——转移节点，用于表示不在正则表中直接相邻的两个 *MST* 之间的转移路径（默认两个 *MST* 在正则表中相邻，即之间没有任何其他节点，那么它们之间存在转移路径）。我们规定，一个 *MST* 后面如果紧接一个或一连串索引节点，那么这些索引节点都修饰这个 *MST*。综上所述，我们使用 *MST* 表示实际的状态，用索引节点的索引表示它所修饰的实际状态拥有哪些转移路径（索引的含义为正则表中 *MST* 的索引）。

2.4 虚转移节点

由于在构建过程中，有时难以判断一个 *MST* 之后需要多少个索引节点，而正则表又是使用数组表示，导致难以确定节点的位置安放和索引节点的索引该如何选取。因此，我们使用一种策略——对于可能有多个转移节点修饰的实际节点之后，我们插入一个虚转移节点，然后维护一个额外的虚转移表数组，此时虚转移节点的索引表示对应的虚转移组的索引，表中含有多个虚转移组，虚转移组含有多个转移节点。实际上，虚转移节点和转移节点都是索引节点，它们的数据组成是相同的。但是它们存在于不同的构建时期，因而有不同的名称。虚转移节点存在于构建过程中，还未整体遍历完表达式时，用于表示多个转移节点。当遍历完成后，进行展开，基于虚转移表和特定算法，将所有虚转移节点替换为它所代表的数个转移节点，同时更新索引到正确的位置。在这个展开过程完成之后，我们认为正则表中的所有索引节点都是转移节点，即不存在虚转移节点了。

2.5 正则表的连接

前文提到，正则表就是状态节点的线性数组。那么，当构建正则表时，不可避免的要对两部分的正则表达式形成的正则表进行连接。

2.6 编译时正则表

由于正则表的起始节点信息和结束节点信息是不那么明显的（需要读取判断），而连接又需要频繁地用到起始节点和结束节点的信息，因此需要一种更擅长连接的结构，当且称为**编译时正则表**。

编译时正则表由五个基本成分组成：

1. **状态节点数组**：对应正则表的状态节点数组，只不过移除了开头的指向第一个节点的转移节点。
2. **起始表**：一个数组，包含数个指向起始节点的索引。
3. **结束表**：一个数组，包含数个指向结束节点的索引。
4. **虚转移表**：一个数组的数组，虚转移表中的每个数组对应一个虚转移节点，其中每个元素共同组成这个虚转移节点展开后的形式，用于在正则表构建完成后提供展开信息，或者供连接使用。

5. **被贯穿标志**：一个布尔值，用于指示这个编译时正则表是否存在起始节点的前驱指向结束节点的后继的转移路径，如果存在，认为被贯穿。

2.7 编译时正则表的首尾相接

考虑两个编译时正则表 A, B ，算法如下：

如果 A 的状态节点数组为空 则输出 B

否则如果 B 的状态节点数组为空 则输出 A 否则

如果 A 的状态节点数组的最后一个节点不是虚转移节点，

添加末尾的虚转移节点，并更新 A 的虚转移表

B 偏移 $\leftarrow A$ 的状态节点数组的元素数量

虚转移表偏移 $\leftarrow A$ 的虚转移表的元素数量

C ：编译时正则表

C 的状态节点数组 $\leftarrow A$ 的状态节点数组为首连接上 B 的状态节点数组为尾

C 的虚转移表 $\leftarrow A$ 的虚转移表为首连接上 B 的虚转移表为尾

新起始表 $\leftarrow B$ 的起始表

令新起始表的每个元素 $index \leftarrow index + B$ 偏移

根据 C 的状态节点数组、虚转移表、起始表和新起始表，使 C 的起始表中指向的所有的

节点后的虚转移节点，其在虚转移表中包含所有指向新起始表中索引的转移节点
遍历 C 的状态节点数组中索引大于等于 B 偏移的节点 $node$ ，如果是虚转移节点，

修正索引为其在新虚转移表中的索引： $node$ 的索引 + 虚转移表偏移

遍历 C 的虚转移表中索引大于等于虚转移表偏移的数组 arr ，更新 arr

中的所有转移节点 $node$ 的索引为： $node$ 的索引 + B 偏移

令 C 的起始表 $\leftarrow A$ 的起始表，令 C 的结束表 $\leftarrow A$ 的结束表

如果满足 A 被贯穿 或者 A 的起始表为空，则将 B 的起始表衔接到 C 的起始表中

如果满足 B 被贯穿，则令 C 的结束表赋值为 B 结束表，否则清空 C 的结束表

新结束表 $\leftarrow B$ 的结束表

令新结束表的每个元素 $index \leftarrow index + B$ 偏移

将新结束表衔接到 C 的结束表的末尾

如果 A, B 都被贯穿，则 C 也被贯穿

输出 C

2.8 状态节点的类型

除去前文所述的普通节点、转移节点 (虚转移节点)。我们着重介绍其余的几种节点。

空转移节点:

空转移节点, 接受一个 ϵ 即可进入此节点, 也就是说, 进入这个节点不需要匹配任何内容, 即此节点的直接前驱节点可以不经过任何条件进入该节点。可以作为正则表中的占位节点, 或者作为转移节点的目标。在正常情况下, 即以此节点为转移目标非必要时, 往往可以忽略此空转移节点。

通配符节点:

通配符节点, 接受一个任意字符即可进入此节点, 也就是说, 只要待匹配的内容不为空, 长度大于等于 1, 即可进入通配符节点。

反字符节点:

反字符节点, 如果一个状态节点的类型是反字符节点, 那么它接受一个与其转移字符不同的字符, 也就是说, 一个反字符节点的转移字符是 a , 那么它可以匹配 $b, c, 1, 2, \#$ 等, 唯独不匹配 a 。

反交集节点:

反交集节点, 用于表示长度为 1 的多个反字符取交集的形式, 反交集节点如果在正则表中相邻, 那么它们共同表示一个长度为 1 的反字符交集, 且反交集节点序列以反字符节点结尾, 最后的反字符节点也包含在交集之中。例如, 要表示 $\sim(a|b|c)$, 可以使用以下顺序表示:

反交集节点(a) $\rightarrow$ 反交集节点(b) $\rightarrow$ 反字符节点(c)

其中 $\rightarrow$ 表示在正则表中左右相邻, $\cdots$ 节点(a) 表示一个转移字符为 a 的某种节点。

阻塞节点:

阻塞节点, 表示所在的分支不可能走通, 等价于删除所在分支。如果它在一个正则表的主干上, 那么这个正则表等价于 ϵ 。

2.9 计算有限正则表的长度

我们将正则表中可以无限循环的路径称为环, 因此闭包运算就是一种构建环的运算。一旦正则表中出现环, 那么它的长度集合就有无限多个元素, 即长度有无限多种可能, 此时长度难以分析, 因此我们只分析无环正则表的长度集合。

分析一个无环正则表的长度, 可以粗略地分为以下几点步骤:

- 枚举所有从起点到终点的简单路径 (无重复节点, 若出现重复节点, 说明存在环)
- 计算所有简单路径的长度
- 将所有长度排序 (可选), 存放在数组或其他数据结构中输出

2.10 反运算修饰的正则结构

在反运算修饰的正则结构中, 需要对正则结构进行分析, 再进行反运算作用后结果的生成。

关键在于, 反运算修饰的正则结构中必须无环, 否则可能出现长度难以判定的情况, 导致正则表的结束位置不确定。

对于空字符串 ε , $\sim\varepsilon = \varepsilon$ 。

对于通配符节点 $\cdot$, $\sim\cdot = E$, E 为阻塞节点。

对于顺序结构, 例如表达式 $R = R_1R_2$, $\sim R = \sim R_1 \cdot^{|R_2|} R_1 \sim R_2$ 。

对于分支结构, 例如表达式 $R = R_1|R_2 \cdots |R_n$, 且 $\forall i$ 满足 $1 \leq i \leq n$, $\forall R_i \neq \varepsilon$, 有 $\sim R = \sim(s_1|s_2| \cdots |s_n) \cdot^{|R|-1} |s_1 \sim S_1|s_2 \sim S_2| \cdots |s_n \sim S_n$, 其中 $s_1, s_2, \cdots, s_n$ 为所有属于 $q_{start}^{R_1}, q_{start}^{R_2}, \cdots, q_{start}^{R_n}$ 中的元素, $S_1, S_2, \cdots, S_n$ 是对于每个 s_i 分支中第一个字符后继的表达式。完成之后, 对 R 进行长度分析, 假设长度集合为 L , 那么 $\forall l \in L$, 将 $\sim R$ 中到达长度为 l 的节点位置添加到结束表中, 即可。

由于闭包运算必定产生环, 因此反运算修饰的正则结构无法含有闭包运算。

一个可能的处理反运算的算法:

内容 $:=$ 反运算修饰的正则结构

反内容 $\leftarrow$ 应用反运算修饰的正则结构的规则(内容)

通配内容 $\leftarrow$ 生成去头的字面节点为通配节点的正则结构(反内容)

反内容 $\leftarrow$ 分析反内容的长度, 并应用更改结束表的规则, 得到的新结构

打包内容 $\leftarrow$ 将后者打包到前者中(反内容, 通配内容)

输出打包内容

需要注意的是, 通配符节点的反运算结果为阻塞节点, 此时需要删除相关的一整个分支。

以及在应用反运算修饰的正则结构的规则的过程中, 转移到通配内容的虚转移节点指向的虚转移组可以是一个仅含有一个空转移节点的数组, 用于专门指代这个虚转移节点的目的地是通配内容, 之后在打包内容生成的过程中重定位。